

Project Name:

Ebook Management System (KitabGhar)

EXPERIMENT 1

Objective 1: Problem Analysis and Project Planning

- **Problem Statement:**

In today's digital era, the popularity of eBooks is rapidly increasing due to their convenience, portability, and cost-effectiveness compared to physical books. However, managing a growing digital library poses significant challenges for both individual users and organizations, including publishers, libraries, and educational institutions. Current systems often lack the flexibility, user-friendliness, and advanced features needed to efficiently organize, access, and share eBooks.

- **Project Scope:**

The scope of the eBook Management System project is to design, develop, and implement a comprehensive platform that addresses the challenges of managing, accessing, and sharing eBooks effectively. The system will cater to individual users, educational institutions, publishers, and libraries, providing a user-friendly and feature-rich solution.

- **Infrastructure Required:**

The infrastructure for the eBook Management System requires a robust setup comprising cloud-based servers for scalability and reliability. Key components include application and database servers with SSD storage and multi-core processors to handle backend operations and user data efficiently. Load balancers ensure high availability and performance during peak usage.

EXPERIMENT 2

Objective 2: Software Requirement Specification (SRS):

1. Introduction :

Purpose & Scope

The eBook Management System is designed to provide an efficient, user-friendly platform for organizing, accessing, and sharing eBooks. This software solution aims to address the challenges faced by individual users, educational institutions, and organizations in managing digital libraries. By leveraging advanced search, synchronization, and collaboration features, the system offers a comprehensive approach to improving the digital reading experience.

Definitions, Acronyms, and Abbreviations

- **SRS:** Software Requirements Specification
- **SSL:** Secure Socket Layer
- **PK:** Primary Key
- **FK:** Foreign Key
- **UAT:** User Acceptance Testing

Modules/Phases:

1. **User Management Module:**
User registration, login, role-based access, and profile management.
Role Types: Individual users, administrators, educators, and institutional accounts.
2. **Upload and Organization Module:**
Upload eBooks, metadata tagging, categorization, and bulk import functionality.
File Support: Compatible with multiple formats (e.g., EPUB, PDF, MOBI).
3. **Admin Management and Analytics Module:**
User account and permissions management, system performance monitoring, and usage analytics.
4. **Collaboration and Sharing Module:**
Sharing eBooks securely, collaborative annotations, and note-sharing tools.
5. **Reading and Accessibility Module:**
Customizable reading interface (themes, font size), bookmarks, annotations, offline reading, and text-to-speech integration.
6. **Security and DRM Module:**
DRM protection, encryption for secure sharing, and compliance with data privacy regulations.

Phase 1: Requirement Gathering and Analysis

Define project scope, objectives, and detailed requirements.

Identify target users and key functionalities.

Phase 2: System Design

Create system architecture, including database schema and module interactions.

Develop UI/UX prototypes for user feedback.

Phase 3: Core Feature Development

Develop foundational modules: User Management, eBook Upload and Organization, and Reading and Accessibility.

Functional Requirements:

- User Registration and Authentication
- Book Upload and Management
- Search and Filtering
- eBook Reader and Preview
- Purchase and Download
- Reading Progress and Syncing
- Ratings and Reviews
- Content Delivery Network (CDN)
- Admin Panel
- Notifications and Alerts
- Accessibility
- Mobile Responsiveness
- Security and Privacy

Non-Functional Requirements:

- Performance
- Availability
- Scalability
- Reliability
- Compliance
- Auditability
- Support and Customer Service

2. Overall Description

Product Perspective

The **Online eBook Management System** (KitabGhar) is a web-based platform designed to provide an efficient and seamless way for users to browse, purchase, read, and manage electronic books (eBooks). The system facilitates interactions between eBook publishers, authors, and readers, enabling them to share, access, and enjoy digital reading materials. It integrates features that allow easy access to eBooks, personalized recommendations, secure transactions, and user-driven content management.

Product Features

- User-friendly interface for customers to manage accounts
 - Library and Recommendations.
 - Search and Browse Functionality.
 - Secure data handling with multi-level authentication.
 - Administrative tools for account creation, management and insights.
-

User Classes and Characteristics

- **Customers:** General users who read eBooks, rate and review books, and manage their personal library.
- **Author/Publisher:** Users who upload and manage their own eBooks for sale or distribution on the platform.
- **Administrators:** Bank personnel who manage user accounts, process customer requests, and monitor system performance.
- **Guest/Visitor:** Unauthenticated users who can browse eBooks but cannot purchase or read full content without registration.

Operating Environment

- **Platform:** Web-based application.
- **Operating System:** Windows/Linux/macOS with a web browser.
- **Browser Compatibility:** Chrome, Firefox, Edge, and Safari.
- **Database:** SQL-based database management system (e.g., MySQL).
- **Security:** SSL encryption for all user sessions.

Design and Implementation Constraints

- The system must support simultaneous multi-user access.
- It should comply with industry standards for security, including encryption and multi-level authentication.
- Response time for all operations must be under 3 seconds.

Assumptions and Dependencies

- The system assumes that users have access to the internet and a web browser.
- The backend database will be continuously available for real-time processing.

EXPERIMENT 3

Objective 3: Data Modeling

1. Data Dictionary

Customer table

Name	Null?	Type
Customer_id (PK)	NOT NULL	INTEGER
Cust_first_name		VARCHAR2(20)
Cust_last_name		VARCHAR2(20)
DOB		VARCHAR2(20)
Gender		VARCHAR2(2)

Login table

Name	Null?	Type
Customer_id (FK)		INTEGER
Password		VARCHAR2(30)
Username		VARCHAR2(30)

Customer Detail table

Name	Null?	Type
Customer_id (FK)	NOT NULL	INTEGER
City		VARCHAR2(20)
State		VARCHAR2(20)
Zip		VARCHAR2(20)
Phone Number		NUMBER(10)
Email id		VARCHAR2(20)

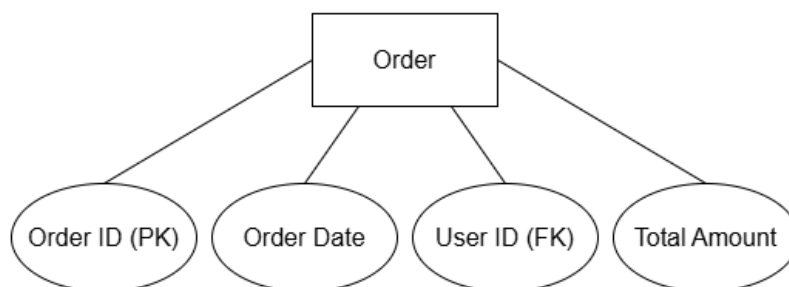
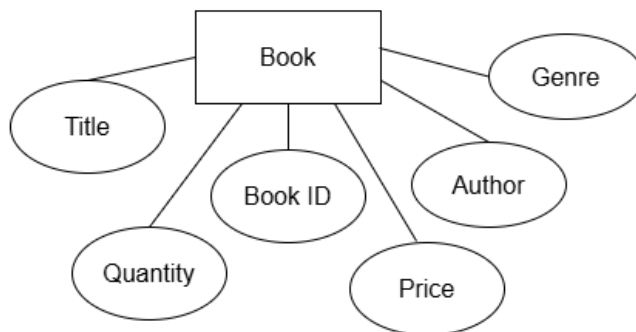
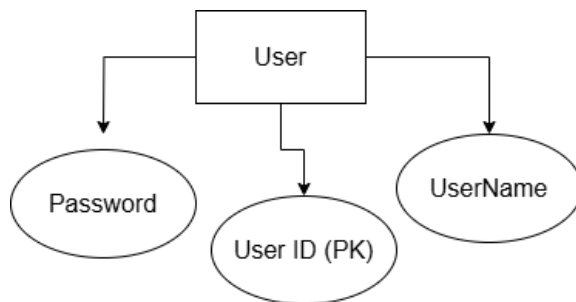
Ebooks Table

Name	Null?	Type
ebook_id	NOT NULL	INTEGER
title		VARCHAR2(30)
author_id		INTEGER
category_id		INTEGER

Categories Table

Name	Null?	Type
category_id	NOT NULL	INTEGER
name		VARCHAR2(20)
description		VARCHAR2(30)

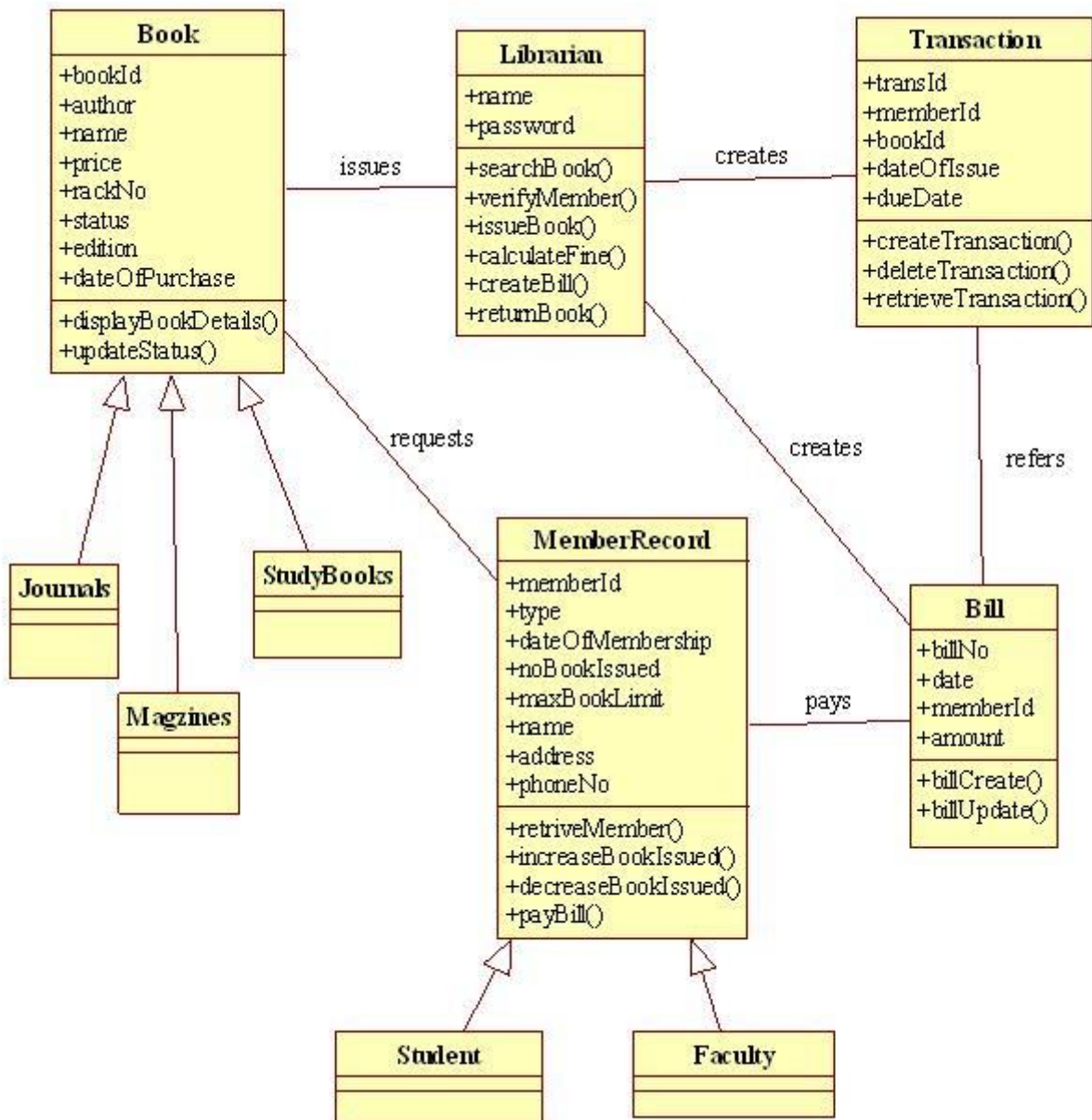
2. Context Level Diagram



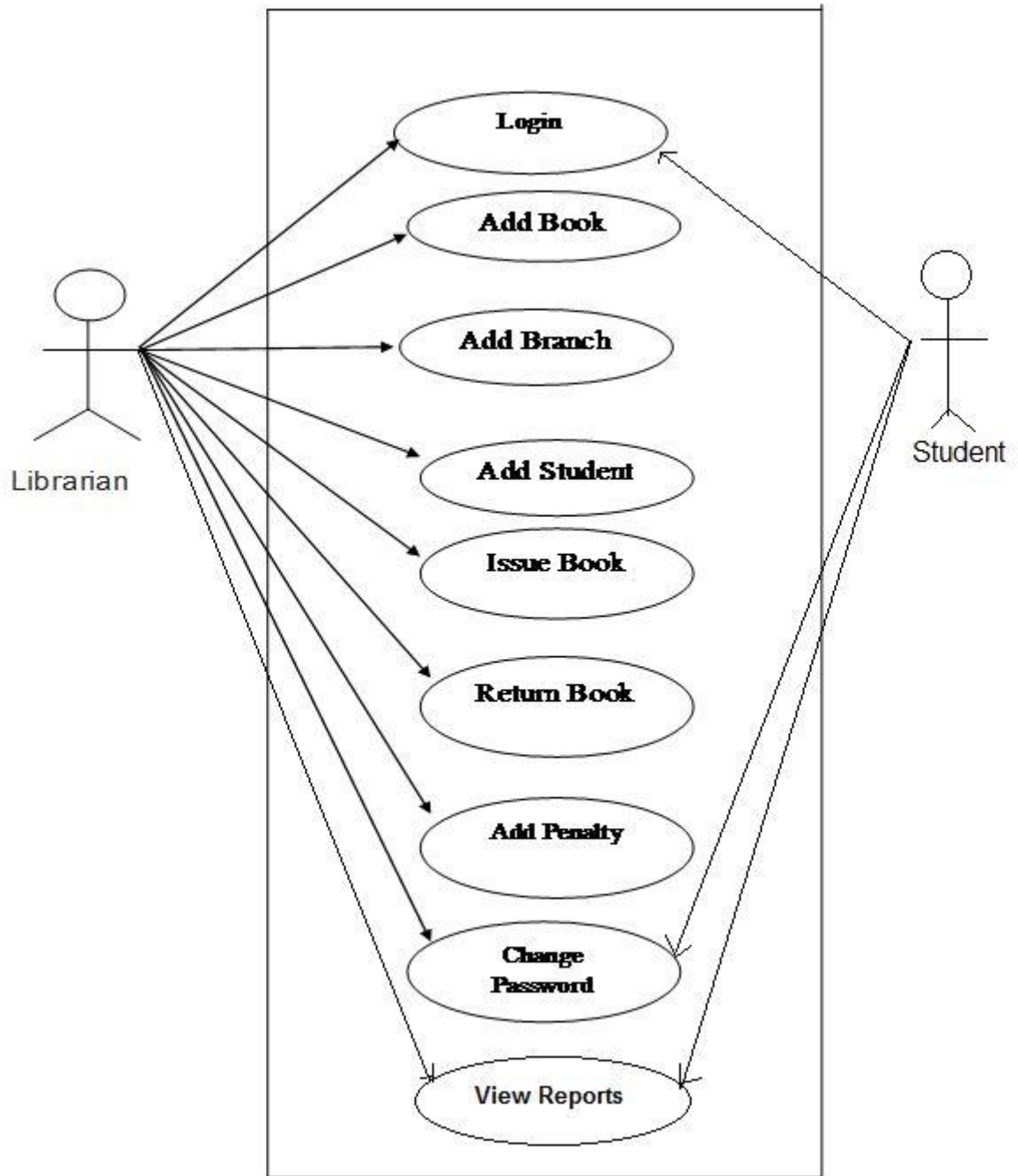
EXPERIMENT 4

Objective 4: Software Designing

1) Class diagram:

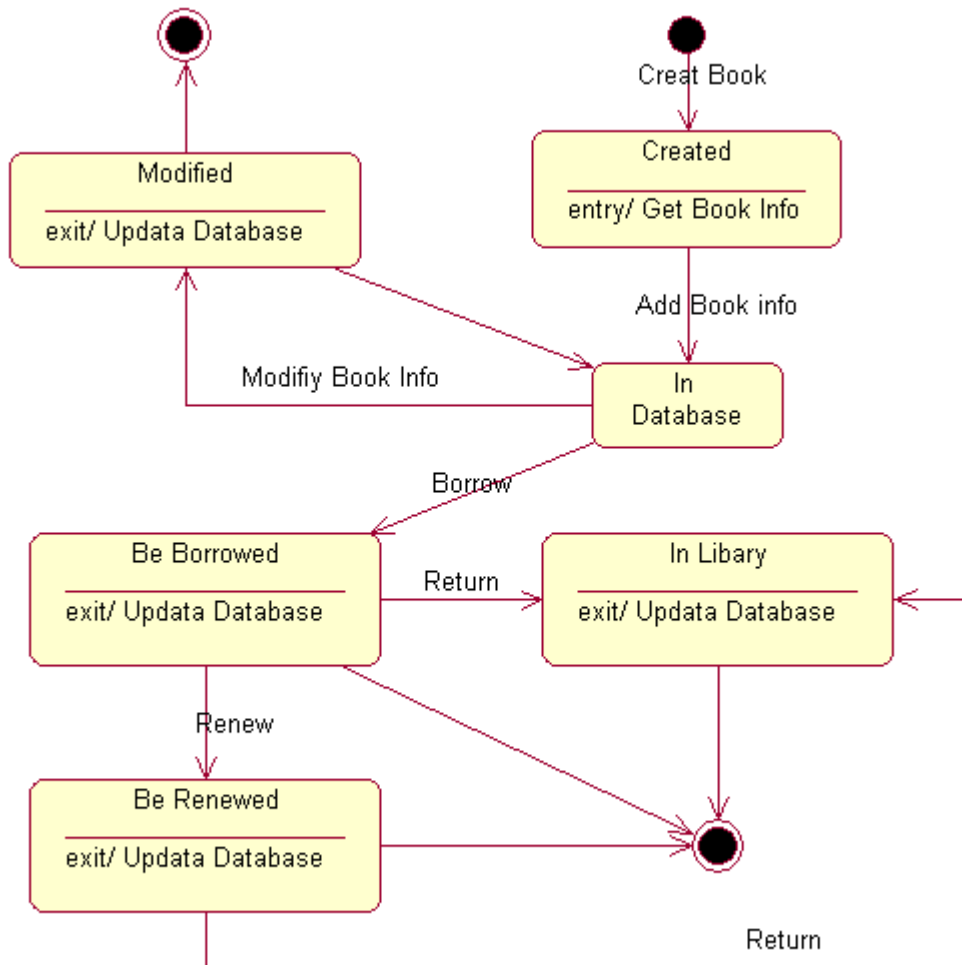


2) Use case Diagram

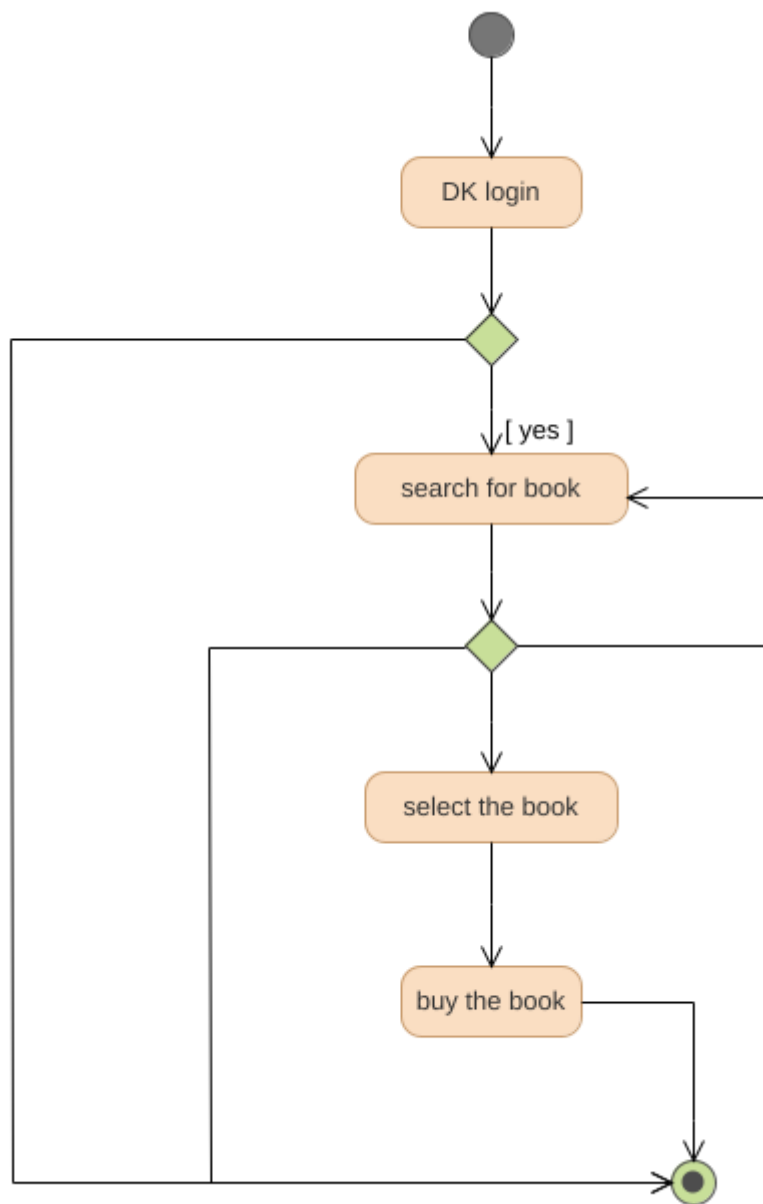


3)Activity Diagram

(3.1) Customer Activity Diagram

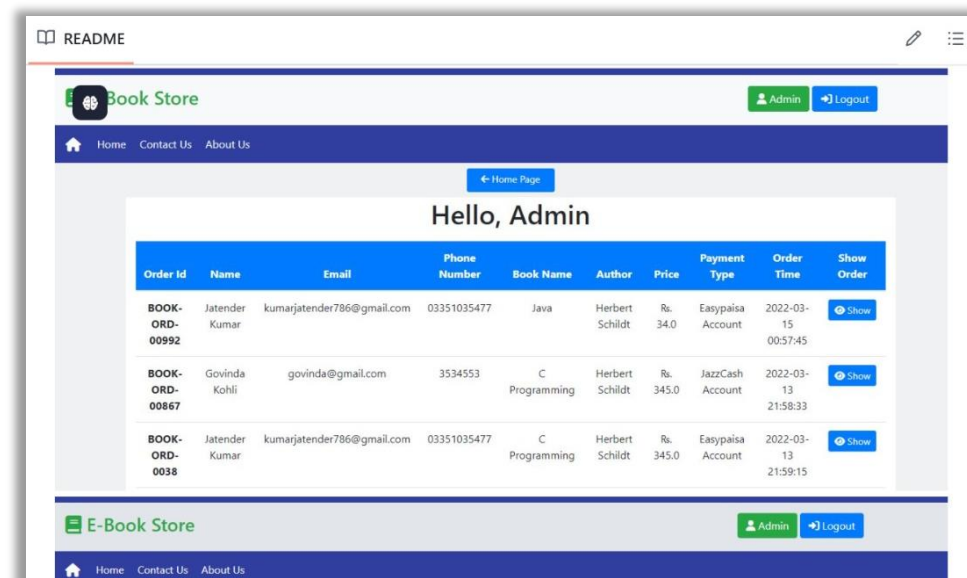
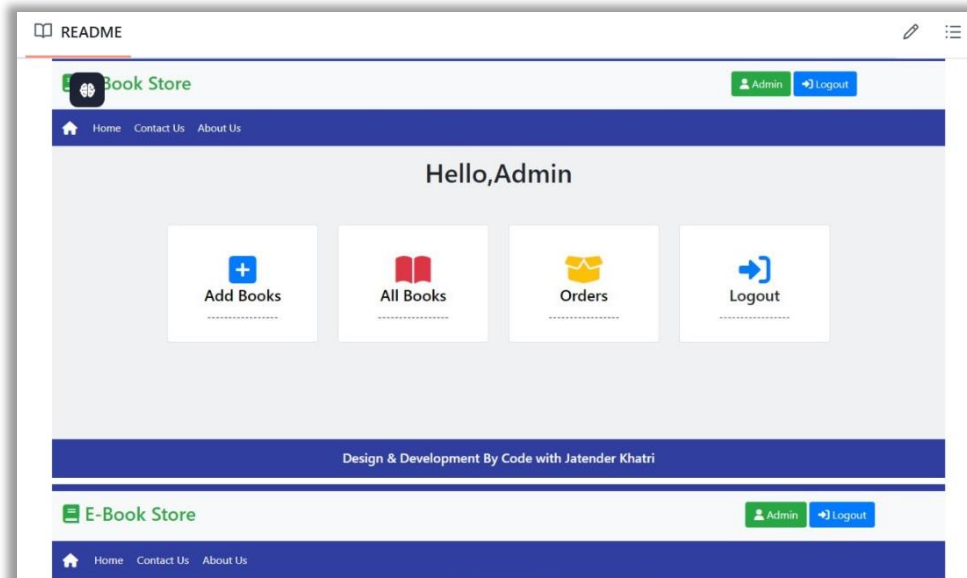
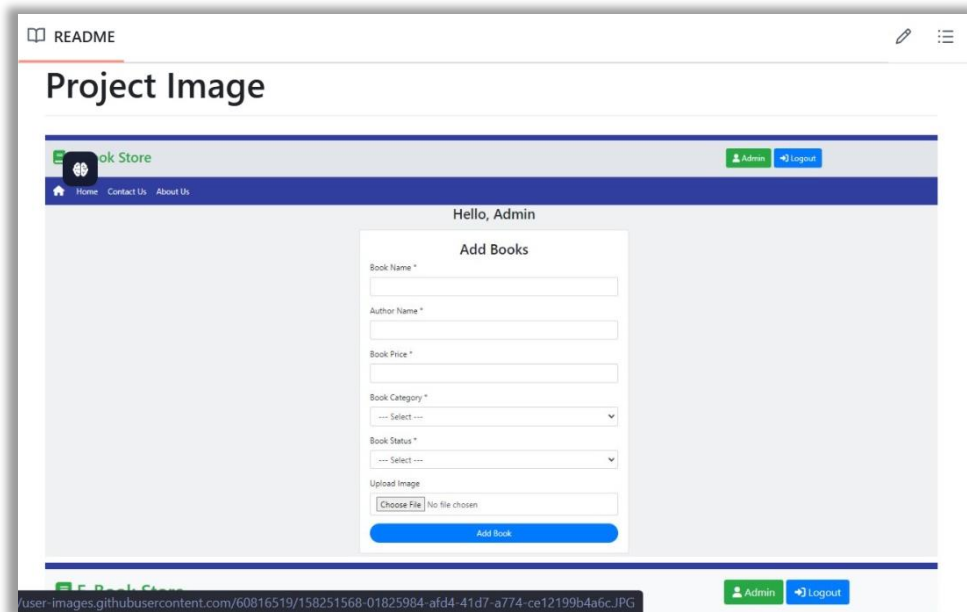


(3.2) Activity Diagram for Administrator



EXPERIMENT 5

Objective 5: Prototype model – Develop the product prototype.



EXPERIMENT 6

Objective : Software Requirement Analysis – Describe the individual Phases/modules of the project and identify deliverables. Identify functional and non-functional requirements.

Software Requirement Analysis

1. User Management

This module handles user registration, authentication, and access control, ensuring that users can securely log in and manage their accounts.

- **Key Features:**
 - User registration and login (email/password or social login).
 - Role management (e.g., admin, author, user).
 - Profile updates and password reset options.
-

2. E-Book Catalog

The catalog serves as the main interface for users to browse, search, and discover e-books.

- **Key Features:**
 - Browse and search with filters (title, author, genre, etc.).
 - Book details with metadata (description, ratings, author).
 - Categorization of e-books by genre or tags.
-

3. E-Book Upload and Management

This module allows admins or authors to add, edit, and manage the e-books available on the platform.

- **Key Features:**
 - Upload books in various formats (PDF, EPUB).
 - Metadata management (title, author, summary).
 - Status control (published, draft, archived).
-

4. Reading Module

Enables users to read e-books directly on the platform with an interactive and customizable reader interface.

- **Key Features:**
 - Online book viewer (PDF, EPUB support).
 - Bookmarking and highlighting.
 - Customizable reading preferences (font, layout, colors).
-

5. Recommendation and Personalization

Offers tailored e-book suggestions based on user activity and preferences.

- **Key Features:**
 - Personalized recommendations (based on history or genre).
 - Similar book suggestions on book detail pages.
-

6. Ratings and Reviews

Provides a space for users to share feedback and ratings on e-books, enhancing community interaction.

- **Key Features:**
 - Leave star ratings and detailed reviews.
 - Display aggregated ratings and top reviews.
 - Admin moderation for reviews.
-

7. Payment and Subscription Management

Handles monetization options, such as subscriptions or purchases for premium content.

- **Key Features:**
 - Subscription plans with tier options.
 - Secure payment gateway integration.
 - Billing history and invoice management.
-

8. Admin Dashboard

A centralized interface for administrators to manage the platform and monitor its performance.

- **Key Features:**
 - Manage users, e-books, and system settings.
 - View analytics (e.g., active users, top books).
 - Resolve user issues or flag content.
-

9. Notifications and Messaging

Keeps users updated on new releases, updates, or personalized alerts.

- **Key Features:**
 - In-app and email notifications.
 - Alerts for recommendations or activity.
 - Notification customization for users.
-

10. Search and Advanced Query Module

Enhances discoverability of content through a powerful and intuitive search engine.

- **Key Features:**
 - Full-text search and autocomplete.
 - Advanced filters (price, language, popularity).
 - Search history for quick navigation.

EXPERIMENT 7

Objective : Testing phase-related documents.

1. Test Plan

- **Purpose:** Outlines the overall testing strategy, scope, objectives, resources, and schedule for testing activities.
 - **Contents:**
 - Testing objectives and scope
 - Test types (e.g., unit testing, integration testing, system testing)
 - Roles and responsibilities
 - Tools and frameworks to be used
 - Test schedule and milestones
-

2. Test Cases

- **Purpose:** Provides detailed step-by-step instructions to validate specific functionalities or requirements.
 - **Contents:**
 - Test case ID
 - Test objective/description
 - Prerequisites (e.g., user must be logged in)
 - Test steps
 - Expected results
 - Actual results
 - Status (Pass/Fail)
-

3. Test Scenarios

- **Purpose:** High-level testing statements that describe what needs to be tested.
 - **Contents:**
 - Scenario ID
 - Scenario description (e.g., "Validate the user login functionality")
 - Test coverage (features/modules impacted)
-

4. Test Data

- **Purpose:** Provides input data used during testing to simulate real-world conditions.
 - **Contents:**
 - Data files (e.g., user credentials, book uploads)
 - Input values for edge cases (e.g., invalid user inputs)
 - Test data mapping with scenarios and cases
-

5. Defect Report

- **Purpose:** Logs and tracks defects identified during testing for resolution.
- **Contents:**
 - Defect ID
 - Description of the issue
 - Steps to reproduce
 - Severity and priority
 - Assigned team member
 - Current status (open, in-progress, resolved)

6. Test Execution Report

- **Purpose:** Summarizes the results of test execution to measure system quality.
- **Contents:**
 - Test case IDs executed
 - Pass/fail status
 - Issues encountered

7. Traceability Matrix

- **Purpose:** Maps test cases to functional and non-functional requirements to ensure complete coverage.
- **Contents:**
 - Requirement ID
 - Associated test case IDs
 - Test status for each requirement

8. Performance Test Report (if applicable)

- **Purpose:** Documents the results of performance and load testing.
- **Contents:**
 - Test environment and configuration
 - Performance metrics (e.g., response times, throughput, memory usage)
 - Bottlenecks identified
 - Recommendations for improvement

9. User Acceptance Test (UAT) Report

- **Purpose:** Documents feedback from end users to validate the system meets business requirements.
- **Contents:**
 - UAT scenarios
 - Test outcomes (approved/rejected)
 - Feedback from users
 - Sign-off from stakeholders

10. Test Summary Report

- **Purpose:** Provides a comprehensive overview of the testing phase, summarizing findings and outcomes.
- **Contents:**
 - Testing objectives and coverage
 - Summary of defects found and resolved
 - Outstanding issues and risks
 - Final test results and sign-off recommendations

EXPERIMENT 8

Objective : Preparation of Software Configuration Management and Risk Management-related documents.

1. Software Configuration Management (SCM) Documents

SCM ensures that changes to the software and its associated artifacts are systematically managed to maintain the integrity and traceability of the project.

Key SCM Documents:

1.1. SCM Plan

- **Purpose:** Describes the processes, tools, and responsibilities for managing changes in the software system.
 - **Contents:**
 - **Objectives:** Goals of SCM (e.g., version control, change management).
 - **Configuration Items:** Items under SCM control (e.g., code, design documents, test cases).
 - **Version Control:** Tools (e.g., Git, SVN) and procedures for version management.
 - **Change Control Process:**
 - Change request submission and approval.
 - Impact analysis and prioritization.
 - **Roles and Responsibilities:** SCM manager, developers, and reviewers.
 - **Auditing and Reporting:** Procedures for configuration audits and reporting changes.
 - **Backup and Recovery Plan:** Ensuring recovery in case of data loss.
-

1.2. Configuration Identification Document

- **Purpose:** Identifies all configuration items and their versions.
 - **Contents:**
 - List of configuration items (e.g., source code files, requirements documents).
 - Unique identifiers for each item.
 - Versioning strategy (e.g., semantic versioning).
-

1.3. Change Request and Control Log

- **Purpose:** Tracks all change requests and their resolutions.
 - **Contents:**
 - Change request ID
 - Description of the change
 - Impact analysis results
 - Approval/rejection status
 - Implementation details
-

1.4. Version Control Log

- **Purpose:** Maintains a log of version history for each configuration item.
- **Contents:**
 - Item name
 - Version number
 - Changes made
 - Author of the changes

- Date of change

1.5. Configuration Audit Report

- **Purpose:** Verifies that configuration items are consistent with their specifications.
- **Contents:**
 - Items audited
 - Audit findings
 - Non-compliance issues and resolutions

2. Risk Management Documents

Risk management involves identifying, assessing, and mitigating risks to ensure the project's success.

Key Risk Management Documents:

2.1. Risk Management Plan

- **Purpose:** Describes the framework for managing risks throughout the project.
- **Contents:**
 - **Risk Identification:** Techniques used (e.g., brainstorming, historical data analysis).
 - **Risk Assessment:**
 - Likelihood of occurrence.
 - Potential impact (low, medium, high).
 - **Risk Response Planning:**
 - Mitigation strategies (e.g., redundancy, training).
 - Contingency plans for critical risks.
 - **Monitoring and Reporting:** Frequency of reviews and reporting mechanisms.
 - **Roles and Responsibilities:** Risk manager, project manager, and team leads.

2.2. Risk Register

- **Purpose:** A centralized document to record and track all identified risks.
- **Contents:**
 - Risk ID
 - Description of risk
 - Likelihood and impact scores
 - Owner (person responsible for monitoring the risk)
 - Current status (active, mitigated, escalated)

2.3. Risk Assessment Matrix

- **Purpose:** Visual representation of risks based on their likelihood and impact.
- **Contents:**
 - Table or chart categorizing risks as low, medium, or high.
 - Mitigation strategies for high-priority risks.

2.4. Risk Response Plan

- **Purpose:** Details specific actions to address high-priority risks.
- **Contents:**
 - Risk description and impact.
 - Chosen mitigation strategy (avoidance, transfer, acceptance, mitigation).
 - Contingency plan for risk realization.

2.5. Risk Monitoring Report

- **Purpose:** Tracks the progress of risk mitigation efforts.
- **Contents:**
 - Status of previously identified risks.
 - New risks discovered during the project.
 - Effectiveness of mitigation strategies.
 - Recommendations for unresolved risks.

EXPERIMENT 9

Objective : Study and use any Design phase CASE tool. Performing the Design by using any Design phase CASE tools

Study and Usage of Any Design Phase CASE Tool

Steps for Using a Design Phase CASE Tool

1. **Choose a CASE Tool:**
 - Examples:
 - **UML Tools:** Visual Paradigm, Lucidchart, StarUML, Enterprise Architect.
 - **ERD Tools** (for database design): MySQL Workbench, dbForge Studio, Microsoft Visio.
 - **Prototyping Tools:** Figma, Adobe XD, Balsamiq.

Performing Design Using a CASE Tool

We will focus on **UML Design** as an example for a software system like an **E-Book Management System**.

Tool: StarUML (or Lucidchart for browser-based tools)

Steps:

1. **Install and Open the Tool:**
 - Download StarUML or open a browser-based CASE tool like Lucidchart.
 - Set up a new project.

2. Use Case Diagram (User Interactions)

- **Objective:** Represent how users interact with the system.
- **Steps:**
 - Create actors like **Admin, User, Author**.
 - Add use cases, e.g., **Login, Search E-Books, Upload E-Books, Read E-Books**.
 - Link actors to their respective use cases using associations.
- **Output:** A Use Case Diagram showing interactions:

mathematica

Copy code

Admin ----> Manage E-Books

User ----> Search E-Books, Rate E-Books

Author ----> Upload E-Books

3. Class Diagram (System Structure)

- **Objective:** Show the system's structure, including entities and their relationships.

- **Steps:**
 - Create classes such as **User**, **EBook**, **Review**, **Subscription**.
 - Define attributes (e.g., User: userID, username, password).
 - Define relationships:
 - Association: A **User** writes **Reviews**.
 - Aggregation: An **EBook** contains multiple **Chapters**.
 - **Output:** A Class Diagram detailing system entities and their relationships.
-

4. Sequence Diagram (Interaction Over Time)

- **Objective:** Illustrate how objects interact in a specific use case (e.g., **Search E-Books**).
 - **Steps:**
 - Define actors (e.g., User).
 - Add objects (e.g., Search Engine, EBook Database).
 - Show the flow:
 - User → Search Engine: "Enter query".
 - Search Engine → EBook Database: "Fetch results".
 - EBook Database → Search Engine: "Return results".
 - Search Engine → User: "Display results".
 - **Output:** A Sequence Diagram demonstrating the interaction flow.
-

5. ER Diagram (Database Design)

- **Objective:** Represent the logical structure of the database.
 - **Steps:**
 - Define entities like **User**, **EBook**, **Review**, **Category**.
 - Define primary and foreign keys.
 - Establish relationships (e.g., a **User** can write multiple **Reviews**, an **EBook** belongs to one **Category**).
 - **Output:** An ER Diagram showing tables and their relationships.
-

Deliverables from CASE Tool:

- Use Case Diagram
 - Class Diagram
 - Sequence Diagram
 - ER Diagram
-

Why Use CASE Tools for Design?

- Ensures clarity in system architecture.
- Facilitates communication among stakeholders.
- Helps detect design flaws early.
- Improves documentation for future maintenance.

EXPERIMENT 10

Objective: Develop test cases for unit testing and integration testing

Test Cases for Unit Testing and Integration Testing

Below are the test cases for unit testing and integration testing, described in plain text for clarity:

1. Unit Testing

Unit testing validates individual components or methods to ensure they perform as intended.

Module: User Authentication

1. Test Case: Validate Correct Login Credentials

- Steps:
 1. Navigate to the login page.
 2. Enter valid credentials.
 3. Click "Login."
- Input: Username: user1, Password: password123.
- Expected Result: User is redirected to the dashboard.

2. Test Case: Handle Invalid Password

- Steps:
 1. Navigate to the login page.
 2. Enter the correct username but an incorrect password.
 3. Click "Login."
- Input: Username: user1, Password: wrongpass.
- Expected Result: Error message "Invalid credentials" is displayed.

3. Test Case: Ensure Required Field Validation

- Steps:
 1. Navigate to the login page.
 2. Leave both fields blank.
 3. Click "Login."
- Input: None.
- Expected Result: Error message "Fields cannot be empty" is displayed.

Module: Search E-Books

1. Test Case: Search for an Existing Book

- Steps:
 1. Navigate to the search bar.
 2. Enter a valid search term.
 3. Click "Search."
- Input: Search Term: Harry Potter.
- Expected Result: List of books containing "Harry Potter" is displayed.

2. Test Case: Handle No Results Scenario

- Steps:
 1. Navigate to the search bar.
 2. Enter a term for a book that does not exist.
 3. Click "Search."
 - Input: Search Term: RandomBook.
 - Expected Result: Message "No results found" is displayed.
-

2. Integration Testing

Integration testing validates interactions between modules to ensure they work together seamlessly.

Scenario: User Authentication and Role-Based Access Control

1. Test Case: Admin Role Access

- Steps:
 1. Login as an admin.
 2. Navigate to the admin panel.
- Input: Role: Admin.
- Expected Result: Admin panel is accessible.

2. Test Case: Restrict User from Admin Panel

- Steps:
 1. Login as a regular user.
 2. Attempt to access the admin panel URL directly.
- Input: Role: User.
- Expected Result: Error message "Access Denied" is displayed.

Scenario: Search Module and Database

1. Test Case: Search Retrieves Correct Data from the Database

- Steps:
 1. Enter a valid search term.
 2. Verify that the displayed results match database entries.
- Input: Search Term: Fantasy.
- Expected Result: Relevant books categorized as "Fantasy" are displayed.

2. Test Case: No Match Found in the Database

- Steps:
 1. Enter a search term that does not exist in the database.
 2. Verify the system's response.
- Input: Search Term: Nonexistent.
- Expected Result: Message "No results found" is displayed.

Scenario: Upload and Catalog Integration

1. Test Case: Uploaded Book Appears in the Catalog

- Steps:
 1. Login as an admin and upload a new book via the admin panel.
 2. Search for the book in the user catalog.
- Input: Book Title: Test Book.
- Expected Result: The newly uploaded book is visible in the catalog.

EXPERIMENT 11

Objective : Areas for future development

1. Advanced Search Features

- Natural Language Processing (NLP): Enable users to search using conversational queries, such as "Find books about space exploration for kids."
- Voice Search: Add support for voice-activated searches to improve accessibility.
- Personalized Search Results: Implement algorithms to tailor search results based on user preferences and behavior.

2. Recommendation Engine

- AI-Driven Recommendations: Introduce machine learning models to recommend books based on reading history, ratings, and genres of interest.
- Collaborative Filtering: Use data from other users with similar tastes to suggest new books.

3. Enhanced User Interface (UI)

- Dark Mode: Add a dark mode option for better user experience during nighttime usage.
- Mobile-Optimized Design: Improve mobile responsiveness to ensure seamless access on all devices.
- Customizable Themes: Allow users to personalize the interface.

4. Integration with External Services

- E-Commerce Integration: Allow users to purchase physical copies of books through integrations with platforms like Amazon or local bookstores.
- Library Integration: Connect with public or university libraries to allow users to borrow e-books directly.
- Social Media Sharing: Enable users to share books, reviews, or achievements (e.g., "Finished reading 5 books this month!") on social platforms.

5. Accessibility Improvements

- Text-to-Speech (TTS): Provide TTS functionality for visually impaired users.
- Language Translation: Offer translation features for e-books in different languages.
- Font Adjustments: Include dyslexia-friendly fonts and other customization options for readers with special needs.

6. Advanced Analytics for Admins

- User Engagement Reports: Offer insights into user activity, such as popular genres, most-read books, and peak usage times.
- Revenue Tracking: For paid features, include detailed analytics on income and user subscriptions.
- Content Performance Metrics: Provide data on the performance of specific e-books or authors.

EXPERIMENT 10

Objective: Develop test cases for unit testing and integration testing

Unit Testing Test Cases:

Test Case Id	Test Name	Objective	Preconditions	Test Steps	Expected Result	Actual Result	Status
TC_E1	Add Book - Success	Verify that a book is added successfully when all required parameters are provided.	- Servlet container running. - Mock dependencies. - Database mock active.	1. Call the doPost method of BooksAdd . 2. Provide valid book details and a valid image.	The book should be added to the database, and the success message displayed.	As expected .	Pass
TC_E2	File Upload	Verify that the uploaded file is stored in the correct directory with the correct name.	- Servlet container running. - Mock file upload support.	1. Call the doPost method of BooksAdd . 2. Provide a valid file part and check the storage path.	File should be stored in the correct directory with the specified name.	As expected .	Pass
TC_E3	Delete Book - Success	Verify that a book is successfully deleted when a valid book ID is provided.	- Servlet container running. - Mock dependencies. - Database mock active.	1. Call the processRequest method of DeleteBookServlet . 2. Provide a valid book ID.	The book should be removed from the database, and the success message displayed.	As expected .	Pass

TC_E4	Sell Old Book - Success	Verify that a book is successfully added as "Old" when all parameters are valid.	- Servlet container running. - Mock dependencies. - Database mock active.	1. Call the <code>processRequest</code> method of <code>SellOldBookServlet</code> . 2. Provide valid parameters (e.g., book name, author, price).	Book is added with the category "Old", status "Active", and success message displayed.	As expected .	Pass
TC_E5	Process Valid Order	Verify <code>processRequest()</code> processes a valid order correctly.	Valid user session, valid cart data.	1. Mock request with valid user and order details. 2. Call <code>processRequest()</code> . 3. Verify redirection to success page.	Redirect to <code>order_success_page.jsp</code> , order details saved successfully.	As expected .	Pass

Integration Testing Test Cases:

Test Case Id	Test Name	Objective	Preconditions	Test Steps	Expected Result	Actual Result	Status
TC_I1	End-to-End Order Processing	Verify end-to-end order processing from adding items to placing an order.	Valid user session, cart items present.	1. Add items to cart. 2. Proceed to checkout. 3. Select payment type. 4. Place order. 5. Verify order summary.	Order is placed, order summary is displayed, and cart is cleared.	As expected .	Pass

TC_I2	Cart Retrieval Integration	Ensure <code>CartDao</code> correctly retrieves cart items for a user and <code>OrderServlet</code> processes them.	User has items in the cart.	1. Mock request with valid user.2. Call <code>processRequest()</code> .3. Verify cart items are retrieved and processed.	Cart items are retrieved and order is processed correctly.	As expected .	Pass
TC_I3	Database Save Integration	Verify <code>BookOrderDao</code> saves order to database and <code>OrderServlet</code> handles the response.	Database is connected and operational.	1. Mock request with valid order details.2. Call <code>processRequest()</code> .3. Verify order is saved in the database.	Order is saved in the database and the success page is displayed.	As expected .	Pass

EXPERIMENT 11

Objective :Develop test cases for various white box and black box testing techniques.

Testing Type	Test Case Description	Expected Output	Actual Output	Pass/Fail
White Box	Verify <code>addBook()</code> function handles both valid and invalid inputs through all code paths.	Valid inputs add the book successfully; invalid inputs trigger appropriate error handling.	As expected	Pass
White Box	Ensure <code>deleteBook()</code> handles different scenarios like invalid IDs and database failures.	Book is deleted for valid IDs; invalid IDs or failure scenarios trigger error messages.	As expected	Pass
White Box	Validate data flow in <code>checkout()</code> from cart to payment processing.	Payment is processed successfully, and order is confirmed; errors are handled for invalid data.	As expected	Pass
Black Box	Test adding a book with valid and invalid data.	Book is added successfully for valid data; error messages for invalid data.	As expected	Pass
Black Box	Verify user login functionality for different credential inputs.	Successful login for valid credentials; error messages for invalid credentials.	As expected	Pass
Black Box	Test cart and checkout process for multiple items and error conditions.	Items are added to cart, and checkout completes without issues; errors are handled gracefully.	As expected	Pass